

~~USB free structure.~~

Module - III

CO3: Illustrate the Working of Processing unit of a Computer System.

Functional Concepts:

To execute a program, the processor fetches one instruction at a time and performs the operations specified. Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered. The processor keeps track of the address of the memory location containing the next instruction to be fetched using the Program Counter PC. After fetching an instruction, the contents of the PC are updated to point to the next instruction. A branch instruction may load a different value into the PC.

Another register in the processor is the Instruction Register IR. To execute an

Instruction, the processor has to perform the following three steps.

- (1) Fetch the contents of the memory location pointed to by the PC. The instruction to be executed in that location, is loaded into IR,

$$IR \leftarrow [PC]$$

- (2) If the memory is byte addressable (each instruction uses 3 bytes), then the contents of PC are incremented by 4.

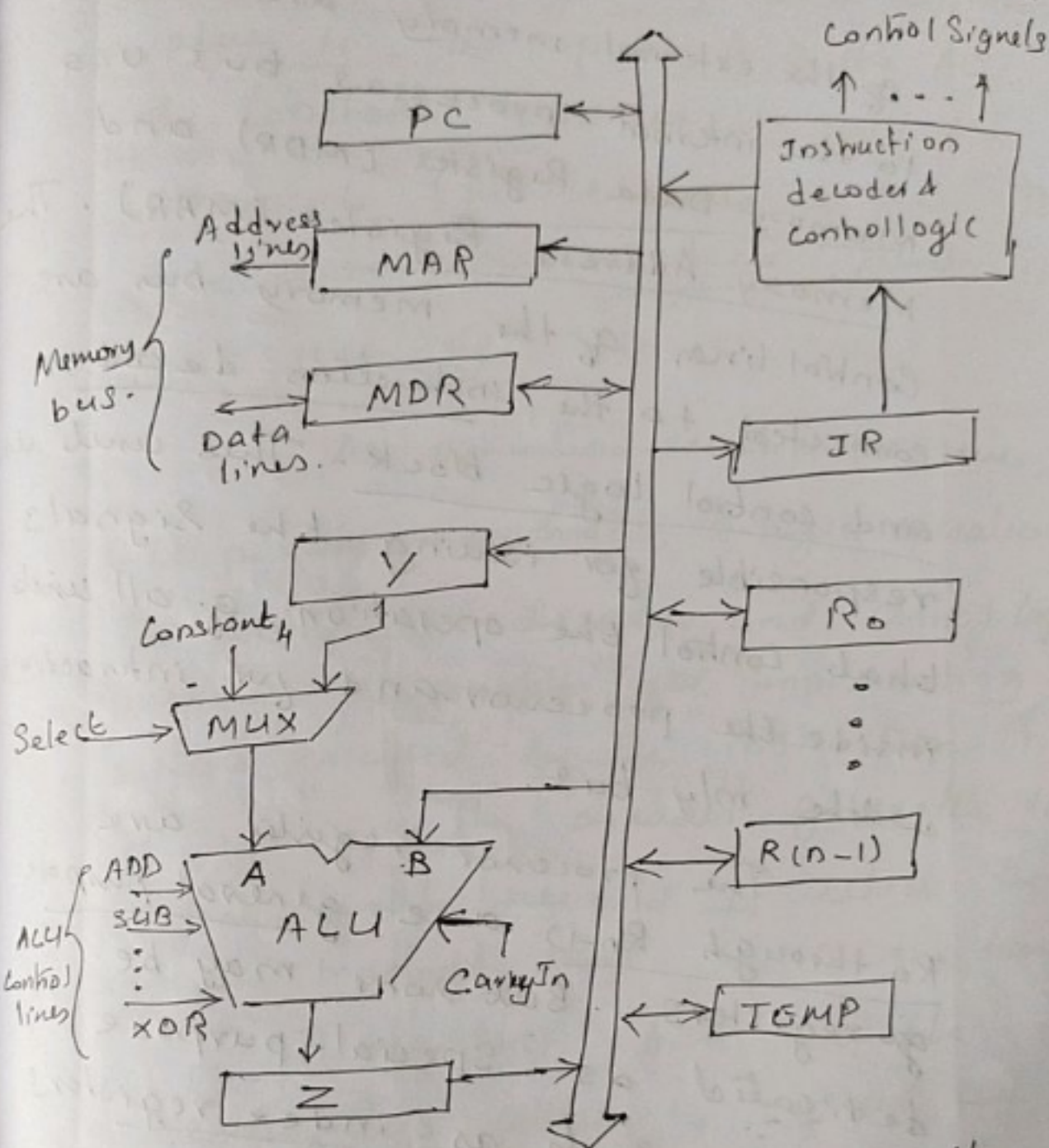
$$PC \leftarrow [PC] + 4$$

- (3) Carry out the actions by the instruction in the IR.

If the instruction occupies more than one word, the step 1 & step 2 are repeated many times to fetch the desired complete instruction.

These two steps are called as fetch phase and the step 3 is the execution phase.

Internal Organization of the processor.



Ans: Single bus Organization of the datapath - inside a processor.

The data lines and address lines of the external memory are connected to the internal processor bus via Memory Data Register (MDR) and Memory Address Register (MAR). The control lines of the memory bus are connected to the instruction decoder and control logic block. This unit is responsible for issuing the signals that control the operation of all units inside the processor and for interacting with the mly bus.

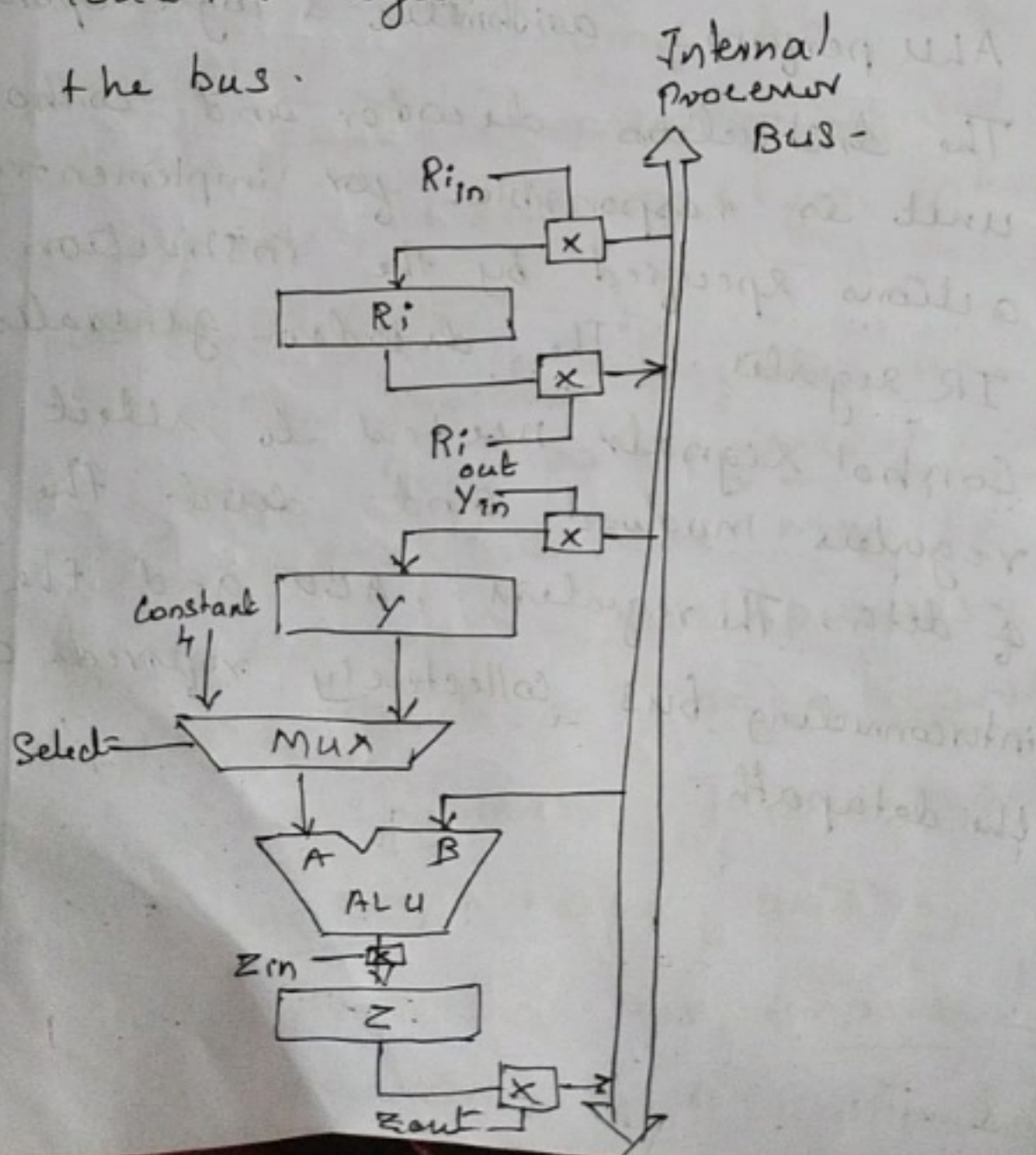
The processor registers are R0 through R15, are general purpose registers. But some may be dedicated as special-purpose registers, such as index registers or stack pointers. Three registers Y, Z and TEMP, are used by register processor for temporary storage during execution of an instruction.

The multiplexer mux selects either the output of register Y or a constant value 4, as input A of the ALU. The constant 4 value is used to increment the contents of PC. The mux control input select, selects the input respectively.

As instruction execution progresses, ALU performs arithmetic & logic operations. The Instruction decoder and control logic unit is responsible for implementing actions specified by the instruction in IR register. The decoder generates the control signals needed to select the registers involved and direct the transfer of data. The registers, ALU and the interconnecting bus collectively referred as the datapath.

Register Transfers

Instruction execution involves a sequence of steps in which data are transferred from one register to another. For each register two control signals are used to place the contents of register on the bus or to load the register with the data on the bus.



The input and output of register R_i are connected to the bus via switches connected by the signals R_{in} and R_{out} respectively. When,

R_{in} set to 1 $\rightarrow$ the data on bus loaded to R_i

R_{out} set to 1 $\rightarrow$ Contents of register R_i placed on the bus.

R_{out} set to 0 $\rightarrow$ bus can be used for transferring data from other registers.

All operations and data transfer within the processor takes place within time periods defined by the processor clock. When two or more clock signals, needed for transfer of data, it is known as multiphase clocking.

Performing an Arithmetic or Logic Operation

ALU, performs arithmetic & logic operations on the two operands applied to A & B inputs. One of the

operand is the O/P of the multiplexer Mux and the other is obtained directly from the bus. The result produced by the ALU is stored temporarily in register Z.

Then, the sequence of operations can be represented, if Y contains the contents in R_1 , B input, of content in R_2 and the result stored in R_3 , as,

1) $R1_{out}, Y_{in}$

2) $R2_{out}, Select Y, Add, Z_{in}$

3) $Z_{out}, R3_{in}$.

The signals whose names are given in any step may remain activated, for the duration of the clock cycle corresponding to that step. All other signals are inactive.

In step 1, the output of register R_1 and the input of register Y are enabled, causing the contents of R_1 are transferred over the bus, to register Y.

In step 2, the multiplexer's select signal is set to select Y, causing the multiplexer to gate the contents of register Y to input A of the ALU. At the same time, contents of R₂ are gated onto the bus and, hence to input B.

The function performed by the ALU depends on the signals applied to its control lines. In this case Add line is set to 1, causing the output of the ALU to be the sum of two numbers at inputs A and B. Sum is loaded to register Z.

In step 3, the contents of register Z are transferred to the destination register R₃.

Fetching a word from the memory.

To fetch a word from memory, the processor has to specify the address of the memory location, where the information

is stored and request a Read operation. The processor transfers the required address to the MAR, whose output is connected to the address lines of the memory bus. At that time, the processor uses the Control lines of the memory bus to indicate that Read operation is needed. The requested data received are stored in register MDR, from where they can be transferred to other registers of the processor.

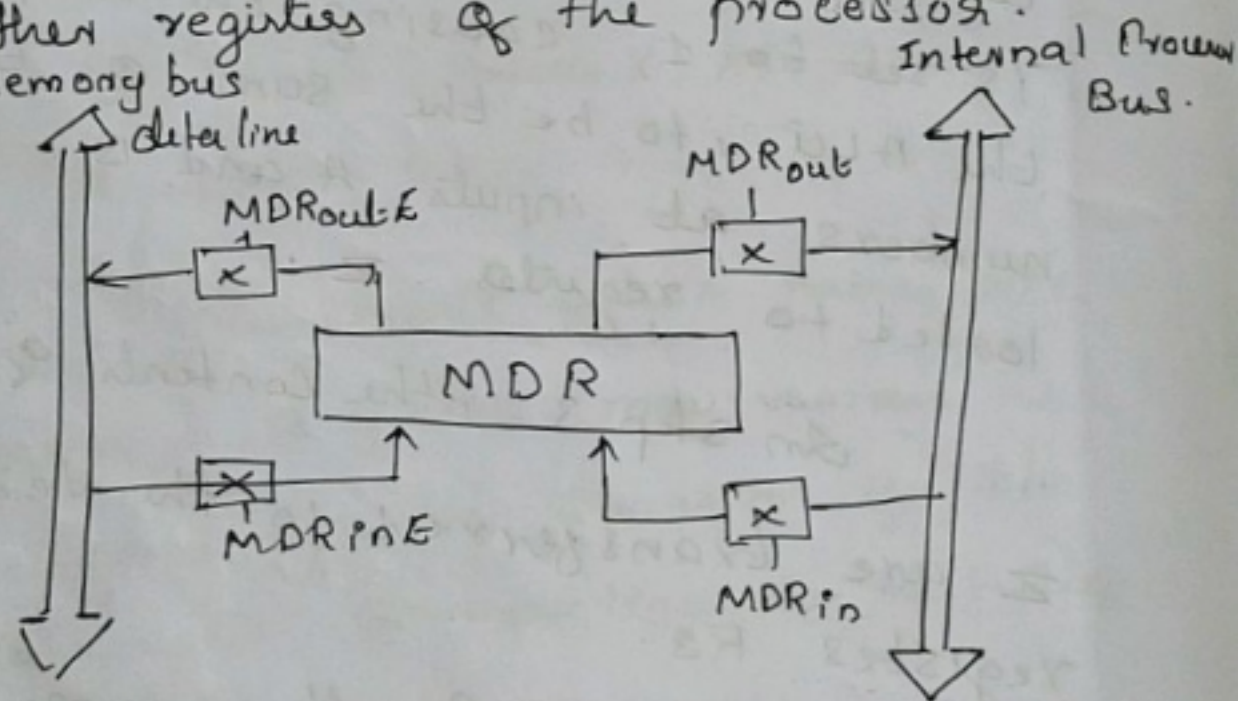


fig: Connection & Control Signals for register MDR

The connection for register MDR is shown in figure. It has four control

These steps, can be described using three steps as
(1) R_{out}, MAR_{in}, Read (3) MDR_{out}, R₂_{in}.
(2) MDR_{in}, WMFC

Signals : MDR_{in} and MDR_{out}, controls the connection to the internal bus, and MDR_{inE} and MDR_{outE} control the connection to the external bus.

During read and write operations the timing of internal processor operations are co-ordinated with the response of the addressed device on the memory bus.

The processor completes one internal data transfer in one clock cycle.

Read operation (example)

Consider the instruction.

MOV (R1), R2.

The actions needed to execute this instruction are:

(1) $MAR \leftarrow [R_1]$

(2) Start a Read operation in the memory bus.

(3) Wait for the MFC response from the memory.

(4) Load MDR from the memory bus.

(5) $R_2 \leftarrow [MDR]$

Memory Function Completed.

Storing a word in Memory

Writing a word in memory is as follows:-

The desired address is loaded into MAR. Then the data to be written are loaded into MDR and a write command is issued.

For executing an instruction,

MOV R₂, (R₁),

- (1) R₁ out, MAR in
- (2) R₂ out, MDR in, write
- (3) MDR_{out} E, WMFC

The processor remains in Step 3 until the memory operation is completed and an MFC response is received, where WMFC is the Control Signal that causes the processor circuitry to wait for the arrival of MFC signal.

Execution of a Complete Instruction

A sequence of elementary operations

required to execute one instruction.

Consider the instruction

ADD (R3), R1,

which adds the instruction contents of a memory location pointed by R3 to register R1. For executing this instruction, it requires following actions:

- (1) Fetch the instruction.
- (2) Fetch the first operand (the contents of memory location pointed to by R3).
- (3) Perform the addition
- (4) Load the result into R1.

These steps can be written as Sequence of Control steps as follows:

<u>Step</u>	<u>Action</u>
(1)	PCout, MARin, Read, Select 4, Add, Zin.
(2)	Zout, PCin, Yin, WMFC
(3)	MDRout, IRin.

- (4) R_{3out} , MAR_{in} , Read
- (5) R_{out} , Y_{in} , $WMFC$.
- (6) MDR_{out} . Select Y , Add, Z_{in}
- (7) Z_{out} , R_{in} , End.

An step 1, the instruction fetch operation is initiated by loading the contents of the PC into the MAR and sending a Read request to the memory. The Select signal is set to Select 4, which causes the multiplexer MUX to select the constant 4. This value is added to the operand at input B, which is the contents of the PC, and the result is stored in register Z. The updated value is moved from register Z back into ~~register~~ PC during Step 2, while waiting for the memory to respond. In step 3, the word fetched from the memory is loaded into IR.

Step 1 through 3 is the fetch phase, which is same for all instructions. The instruction decoder circuit interprets the contents of the IR at the beginning of step 4. This enables the control circuit to activate the control signals for steps 4 through 7, which is the execution phase. The contents of Register R_3 are transferred to MAR in step 4, and a memory read operation is initiated. Then the contents of R_1 are transferred to register Y in step 5, to prepare for the addition operation.

When the read operation is completed, the memory operand is available in register MDR, and the addition operation is performed. The contents of MDR are gated to the bus, and then also to the B input of the ALU, and register Y is selected as second input to the ALU by choosing select Y . The sum is stored in register Z , then transferred to R_1 in step 7. The End signal causes a new instruction fetch cycle to begin.

Branch Instruction

A branch instruction replaces the contents of the PC with the branch target address. This address is usually obtained by adding an offset x , which is given in the branch instruction, to the updated value of the PC.

The below steps gives a control sequence of an unconditional branch instruction.

<u>Step</u>	<u>Action</u>
(1)	PCout, MARin, Read, Select 4, Add, Zin.
(2)	Zout, PCin, Yin, WMFC
(3)	MDRout, IRin
(4)	Offset-field-of-IRout, Add, Zin
(5)	Zout, PCin, End.

Processing starts as usual, with the fetch phase. This phase ends when the instruction is loaded into IR, in step 3. The offset value is extracted from the IR, by the instruction decoding circuit. Since the value of the updated PC is already available in register Y, the offset X is gated onto the bus in step 4, and an addition operation is performed. The result of the target address is loaded into the PC in step 5.

The offset X is used in a branch instruction is usually the difference between the branch target address and the address immediately following the branch instruction.

Now, consider a conditional branch. In this case, we need to check the status of the condition code before loading a new value into the PC.
eg: For a branch-on-negative (Branch <0) instruction, step 4, in the above sequence are replaced by,

Opert - field - Op - IRout, Add, Zin, if $N=0$ then

Thus, if $N=0$, the processor returns to step 1 immediately after step 4. If $N=1$, step 5 is performed to load a new value into the PC then performing the branch operation.

Multiple-Bus Organization: